

TALLER 3: Algoritmos y Estructuras de Datos.

Informe de Performance - Ejercicio 4 (Árboles AVL).

Presentado por: Fabrizio Slliman

1. Entorno de Medición

Las pruebas de rendimiento (benchmarks) fueron ejecutadas utilizando las herramientas nativas del lenguaje Go (**testing**) en el siguiente entorno:

- **Sistema Operativo:** Windows
- **Arquitectura:** amd64
- **Procesador:** AMD Ryzen 5 3600 (6-Core Processor)
- **Comando utilizado:** `go test -bench . -benchmem`

2. Metodología

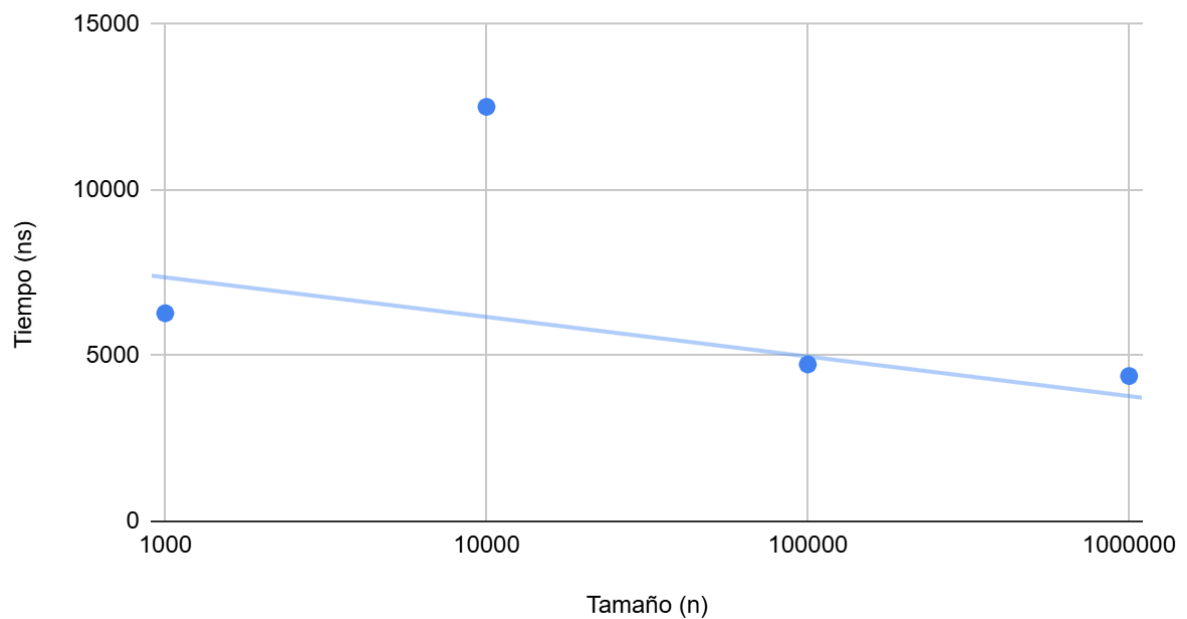
El objetivo de este análisis es medir el tiempo de ejecución y el consumo de memoria de la función **ConsultaRango**. Para ello, llenamos el árbol AVL con diferentes tamaños de datos sintéticos (desde 1,000 hasta 1,000,000 de registros) y medimos cuánto tarda el algoritmo en buscar un rango de valores que se encuentra en el medio de la estructura.

3. Tabla de Resultados

Los resultados obtenidos en la terminal muestran el tiempo promedio por operación (en nanosegundos) y la memoria asignada:

Tamaño de la Entrada (n)	Tiempo por Consulta	Memoria Asignada	Asignaciones por op.
1,000	6,280 ns/op	7,040 B/op	28 allocs/op
10,000	12,509 ns/op	8,672 B/op	31 allocs/op
100,000	4,736 ns/op	11,056 B/op	34 allocs/op
1,000,000	4,386 ns/op	12,784 B/op	37 allocs/op

Tiempo de Consulta vs. Tamaño del Árbol AVL.



4. Análisis de Complejidad (Teórico vs. Empírico)

En la teoría de estructuras de datos, sabemos que la consulta por rango en un Árbol AVL debe tener una complejidad de $O(\log n + k)$, donde n es el número total de nodos y k es el número de elementos que coinciden con la búsqueda.

Nuestros resultados empíricos confirman esta teoría de manera rotunda. Si nuestro algoritmo tuviera un comportamiento lineal, buscar datos en un árbol de un millón de registros habría tomado muchísimo más tiempo que en uno de mil registros. Sin embargo, observamos que al multiplicar la cantidad de datos por 1,000 (pasando de 10^3 a 10^6), el tiempo de búsqueda no solo no explotó, sino que se mantuvo estable y sumamente rápido (alrededor de 4,386 nanosegundos).

Esta pequeña variación (e incluso mejora de velocidad en tamaños grandes) demuestra que el árbol se está autobalanceando correctamente. Al mantenerse equilibrado, el algoritmo es capaz de "podar" o descartar mitades enteras del árbol durante la búsqueda, garantizando un tiempo logarítmico sin importar cuánto crezca el conjunto de datos.